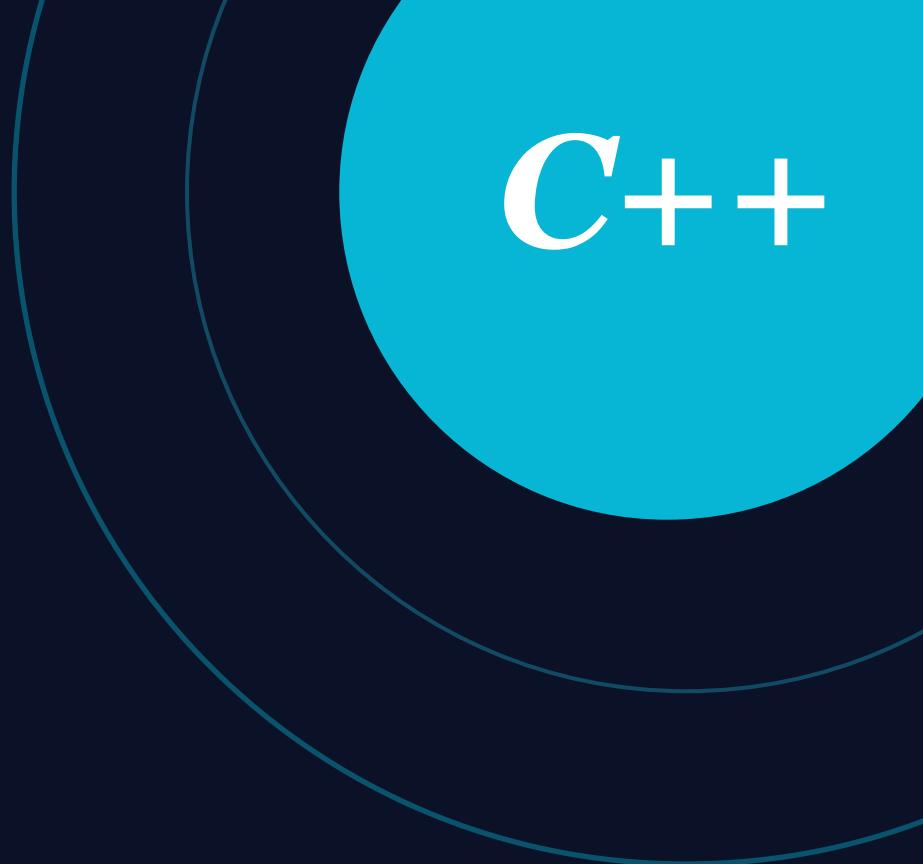




C++

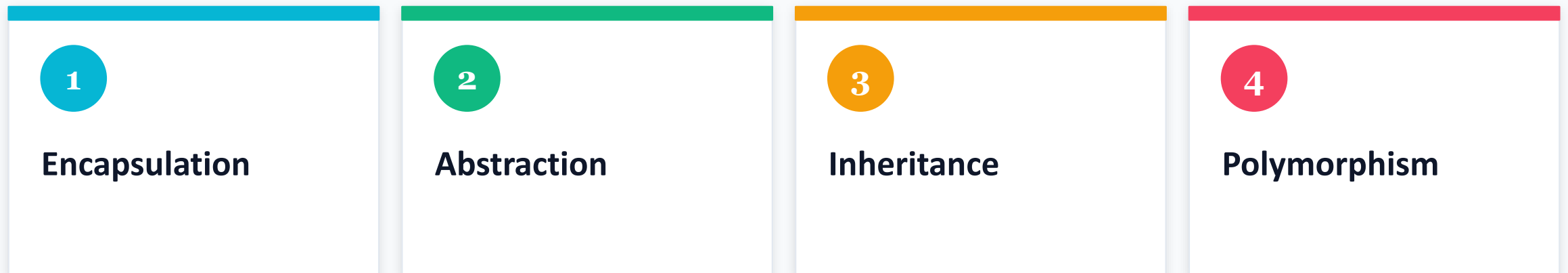
OBJECT ORIENTED PROGRAMMING

The Pillars of OOP

Object-Oriented Programming (OOP)

OOP is a programming paradigm that organizes software design around objects rather than around functions and logic. Objects are self-contained units that bundle data (attributes) and behaviour (methods). C++ supports OOP through classes, objects, and four core pillars.

THE FOUR PILLARS OF OOP



Classes & Objects

CLASS

A user-defined data type that serves as a blueprint for creating objects. It groups data members (attributes) and member functions (methods) under one name.

OBJECT

An instance of a class and is an allocated entity in memory. Each object has its own copy of the class's non-static data members and can invoke its member functions.

Class : Blueprint • Object : Instance

Encapsulation

Encapsulation is the wrapping of data members and the member functions that operate on them into a single unit (the class), together with control of access to that data through access specifiers. Typically, the data is kept private and exposed only through public methods.

KEY BENEFITS

- Data hiding — protects internal state from external misuse
- Modularity — clear interface between user and implementation
- Maintainability — internals can change without breaking users
- Validation — setters can enforce invariants on writes

Access Specifiers — public, private, protected

Access specifiers control where the members of a class can be accessed from. They are the foundation of encapsulation in C++.

public

Members are accessible from anywhere i.e. inside the class, by derived classes, and from outside code through an object.

private

Members are accessible only inside the same class and by friend functions/classes. The default access mode for class members in C++.

protected

Members are accessible inside the same class and by classes derived from it, but not from outside code.

Abstraction

Abstraction is the act of representing essential features of an entity without including the underlying implementation details. It lets the user interact with an object through a simplified interface while complexity is hidden inside.

TYPES OF ABSTRACTION

Data Abstraction

Showing only essential data attributes to the outside world while hiding the implementation of the data. Achieved in C++ using classes with private/protected data and public methods.

Control / Process Abstraction

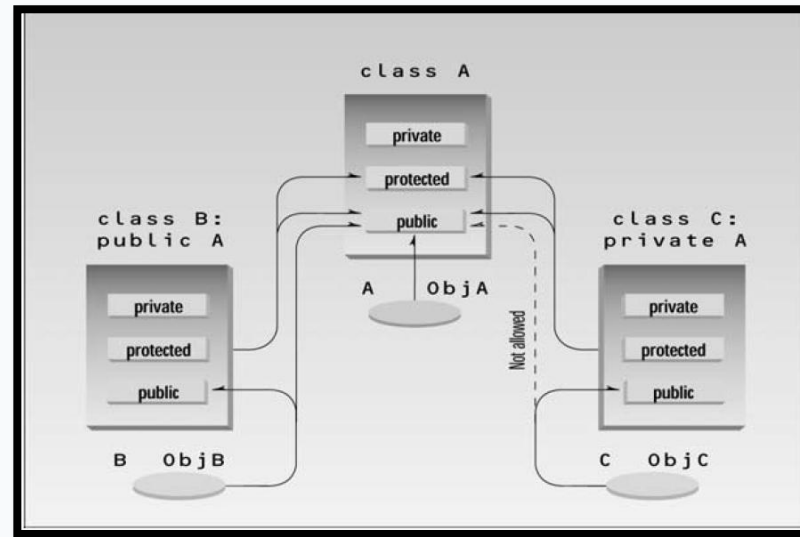
Hiding the details of how an operation is carried out, exposing only what it does. Achieved through functions and abstract classes (pure virtual functions).

How achieved in C++ : access specifiers (private/protected) • abstract classes • pure virtual functions • header/implementation separation.

Inheritance

Inheritance is the mechanism by which one class (the derived / child / sub class) acquires the properties and behaviour of another class (the base / parent / super class), and may extend or override them. It models an "is-a" relationship and promotes code reuse.

ACCESS MODE



FIVE TYPES OF INHERITANCE

Single

Multiple

Multilevel

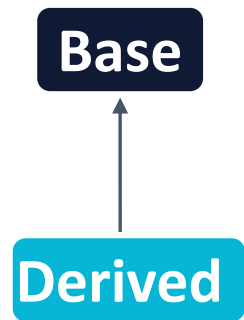
Hierarchical

Hybrid

Five Types of Inheritance

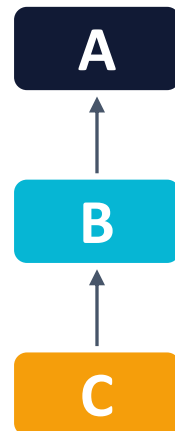
Single

One derived class inherits from one base class.



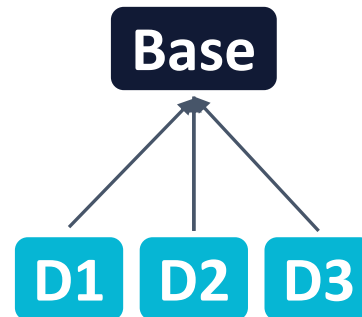
Multilevel

A class is derived from another derived class.



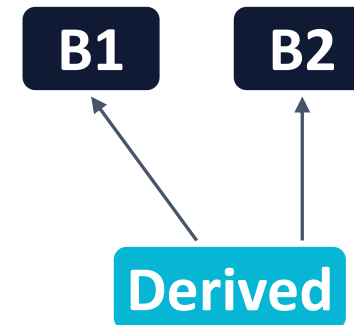
Hierarchical

Two or more classes inherit from a single base class.



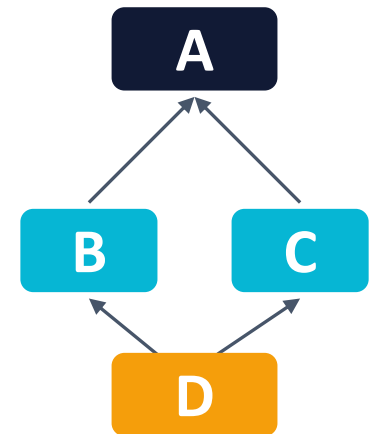
Multiple

A derived class inherits from two or more base classes.



Hybrid

Combination of two or more types.



Polymorphism — Overview

Polymorphism ("many forms") is the ability of a single interface, name, or operator to behave differently depending on the type or context of its operands. In C++, it lets one symbol represent multiple implementations resolved at compile or run time.

Polymorphism

```
graph TD; A[Polymorphism] --> B[Compile-time / Static]; A --> C[Run-time / Dynamic];
```

Compile-time / Static

Resolved by the compiler (early binding).

Mechanisms: function overloading, operator overloading, constructor overloading, templates.

Run-time / Dynamic

Resolved during program execution (late binding) using the vtable.

Mechanism: via virtual functions and base-class pointers/references.

Compile-time Polymorphism — Function Overloading

Function overloading is defining two or more functions with the same name in the same scope but with different parameter lists (number, type, or order of parameters). The correct version is selected by the compiler based on the arguments at the call site.

RULES

- Functions must differ in parameter list
- Return type alone is NOT enough to overload
- Resolved at compile time (early binding)

Operator & Constructor Overloading

OPERATOR OVERLOADING

Giving an existing C++ operator (+, -, ==, <<, etc.) a user-defined meaning when applied to operands of a class type. Defined as a special member or non-member function named `operator<symbol>`.

CONSTRUCTOR OVERLOADING

Defining multiple constructors in the same class with different parameter lists. The compiler selects which one to call from the arguments used to create an object.

Run-time Polymorphism — Function Overriding

Function overriding is redefining a base-class member function in a derived class with the same name, return type, and parameter list. When the base function is declared virtual, the call is resolved through the vtable at run time based on the actual object type — late binding.

KEY KEYWORDS

- virtual — enables dynamic dispatch from base
- override — compiler-checked override in derived
- = 0 (pure virtual) → abstract class
- Use base pointer / reference to invoke

Overloading vs Overriding

Both reuse a function name, but they happen in different scopes, are bound at different times, and serve different purposes.

Aspect	Function Overloading	Function Overriding
Scope	Same class / same scope	Across base & derived classes
Parameter list	Must differ (count / type / order)	Must be identical (same signature)
Inheritance	Not required	Required
Binding	Compile-time (static / early)	Run-time (dynamic / late)
Polymorphism type	Compile-time / static polymorphism	Run-time / dynamic polymorphism
Keyword	None special	virtual (base) + override (derived)
Purpose	Same name for similar operations on different inputs	Replace inherited behaviour with a specialized one

Static Members — Variables & Functions

STATIC DATA MEMBER

A class member declared with the static keyword. Only ONE copy exists for the entire class. It is shared by all objects, not duplicated per instance. It must be defined once outside the class.

STATIC MEMBER FUNCTION

A member function declared static. It belongs to the class itself and can access only static data and other static functions. Called using the class name.

Friend Functions

A friend function is a function that is not a member of a class but is granted access to the class's private and protected members through the friend keyword in the class declaration.

PROPERTIES

- Not a member — has no this pointer
- Called like an ordinary function, not via an object
- Friendship is granted, not taken — and is not inherited
- Friendship is not transitive (A's friend $\neq$ B's friend)
- Often used for operator<< / operator>> overloads

Takeaways

Classes & Objects

A class is a blueprint; an object is an instance allocated in memory.

Abstraction

Show essential interface (data / control), hide implementation.

Polymorphism

Compile-time (overloading) vs run-time (overriding via virtual functions).

Encapsulation

Wrap data + methods and control access via public, private, protected.

Inheritance

Reuse and extend behaviour: single, multiple, multilevel, hierarchical, hybrid.

Static & Friend

Static = class-level shared members. Friend = external access to private state.